koroteevmv / ML_course
Code Issues 2 Pull requests Agents Actions Projects Security and quality Insights
Type / to search
Files
main
Go to file
ML4.1 metrics
ML4.2 threshold
ML4.3 cross-validation
ML4.4 diagnostics
ML4.5 hyperparameters
ML5.1 data integration
ML5.2 numeric features
ML5.3 categorical features
ML5.4 target
ML5.5 EDA
ML6.1 clustering
ML6.2 pca
ML7.3 text
ML7.4 images
misc
LICENSE
README.md
README_template.md
ML_course / ML5.4 target /
Add file
koroteevmv Update README.md af0e41e · last year History

Name	Last commit message	Last commit date
..		
data	Картинки распределены по папкам	2 years ago
img	Продолжена методичка ML54	last year
README.md	Update README.md	last year

Работа с целевой переменной

Цель работы

Познакомиться с основными приемами обработки данных в отношении к целевой переменной: дискретизация, отбор признаков, устранение дисбаланса классов.

Содержание работы

1. Загрузите первый датасет для регрессии и познакомьтесь с его структурой.
2. Постройте простую модель регрессии и оцените ее качество.
3. Отберите признаки, наиболее сильно влияющие на значение целевой переменной.
4. Постройте модель на оставшихся данных и оцените ее качество.
5. Загрузите второй датасет для регрессии и постройте распределение целевой переменной.
6. Сгруппируйте значения целевой переменной в категории. Постройте получившееся распределение.
7. Загрузите датасет для классификации. Постройте распределение целевой переменной.
8. Разделите датасет на тестовую и обучающую выборки, постройте и оцените baseline модель классификации.
9. Постройте ту же модель с применением весов классов. Сравните ее качество, сделайте выводы.
10. Выравняйте распределение классов путем оверсемплинга с повторением

Методические указания

В предыдущих работах мы занимались предобработкой разных типов данных, ориентируясь на сами характеристики этих данных: их распределения, аномалии, шкалы и так далее. То есть, мы ориентировались на информацию, которую содержат эти переменные изолированно. Однако, для целей моделирования нас в первую очередь интересует то, как тот или иной признак влияет на значение целевой переменной.

В данной работе мы познакомимся с основными операциями обработки данных, которые принимают во внимание соотношение признаков и целевой переменной. Среди них преобразования самой целевой переменной, отбор признаков, работа с несбалансированными датасетами.

Отбор признаков по важности

В этой работе мы будем практиковаться на наборах данных с сайте OpenML. Это один их крупных открытых репозиториев датасетов, моделей и алгоритмов машинного обучения, наподобие Kaggle, отличающийся удобным поиском с возможностью фильтрации по датасетам. Что еще более удобно, интеграция с этим репозиторием встроена в библиотеку sklearn, так что можно воспользоваться одной функцией для загрузки датасета. Ее можно импортировать из пакета datasets:

```
from sklearn.datasets import fetch_openml
```

Для загрузки датасета нам понадобится указать его имя. Если вы ищете датасет на сайте, то его имя указано в заголовке страницы датасета. Для первого примера мы возьмем датасет mtp, содержащий фармокологические данные. Этот датасет подходит для наших целей: в нем довольно много признаков, не все и которых очень показательны для значения целевой переменной. Кроме названия следует указать версию датасета:

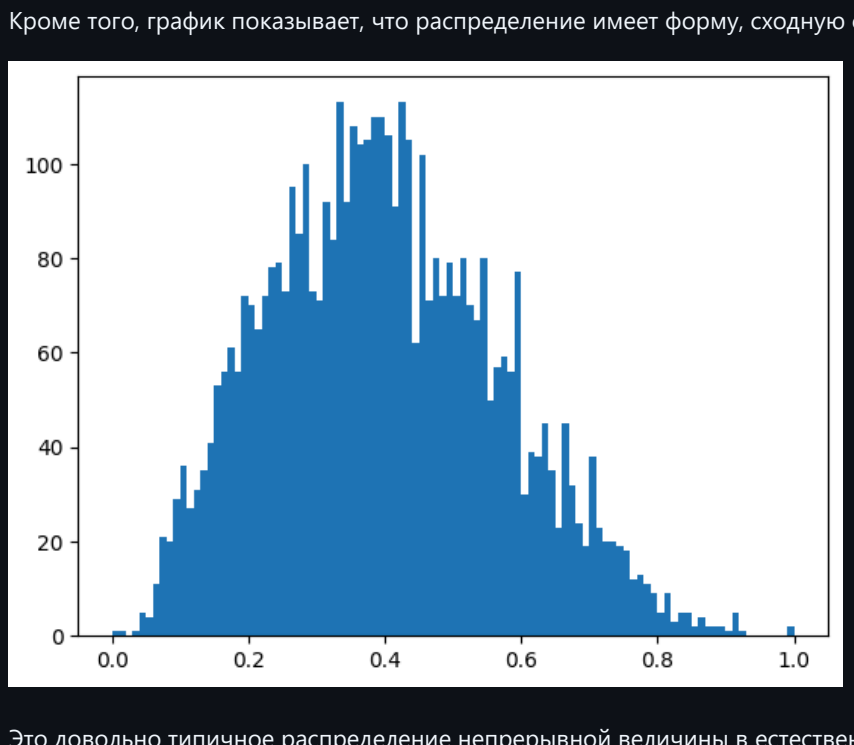
```
df = fetch_openml("mtp", version=1)

df.data.head()
```

Теперь мы можем оперировать этим датасетом как и любым другим. У датасетов OpenML общий интерфейс: они представляют собой словарь со стандартными ключами. В частности, признаки хранятся по ключу data, а значения целевой переменной - по ключу target. Таким образом, наш датасет уже не нужно разделять на X и y.

Давайте построим распределение целевой переменной:

```
plt.hist(df.target, 100)
_ = plt.plot()
```

Мы видим, что целевая переменная представляет собой численное значение, что определяет задачу моделирования как регрессию. Кроме того, график показывает, что распределение имеет форму, сходную с нормальным:



Это довольно типичное распределение непрерывной величины в естественных данных. Ничего особенно примечательного здесь нет. Мы его построили справочно, более подробно с распределением целевой переменной будем работать в следующих пунктах.

Построение базовой (baseline) модели

Для того, чтобы оценивать эффективность тех или иных методов обработки данных, желательно понимать, как они влияют на эффективность обучаемых моделей. Для этого до начала любых преобразований данных нужно построить базовую простую модель и оценить ее эффективность. Такая базовая модель часто называется простой baseline или базовой моделью. Такая модель позволяет выбрать те способы изменения исходного датасета, которые увеличивают его предсказательную силу, то есть работают на увеличение точности моделей. Именно с бейзлайном мы будем сравнивать эффективность моделей после преобразования данных.

В любом случае, нам потребуется разделить выборку на обучающую и тестовую для несмещенного оценивания уровня эффективности модели. Для еще более точной оценки, можно использовать перекрестную проверку, можете сделать это самостоятельно, мы же воспользуемся обычным разбиением:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(df.data, df.target, test_size=0.25, random_state=42)
```

В качестве базовой модели лучше выбрать простую (вычислительно) модель, которая, желательно, обладает высокой интерпретируемостью. Для этого лучше всего подходят линейные модели и деревья решений. В данном случае, воспользуемся моделью линейной регрессии:

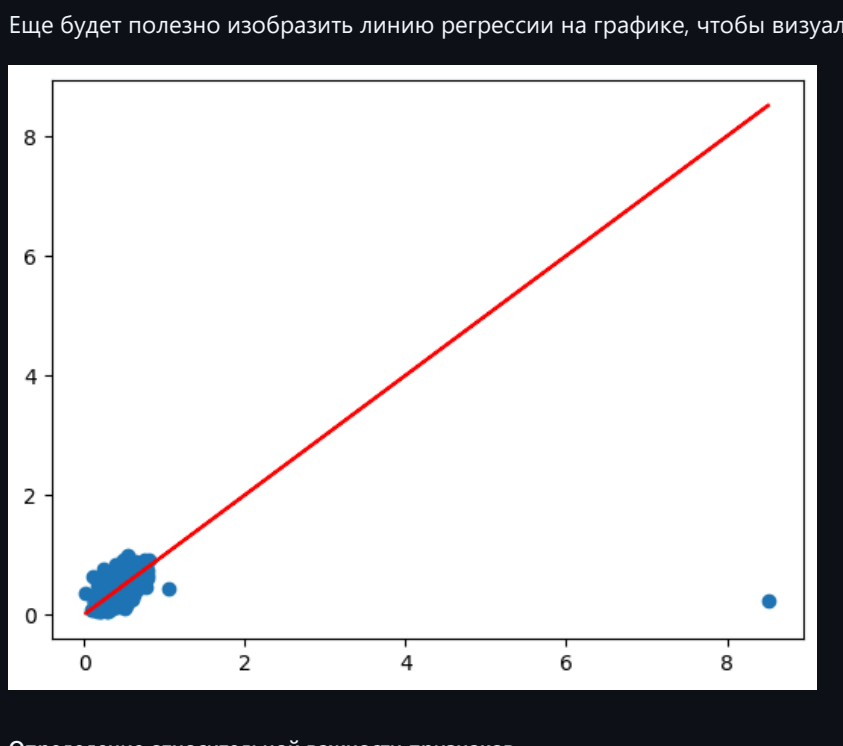
```
baseline = LinearRegression()
baseline.fit(X_train, y_train)
bl_score = baseline.score(X_test, y_test)
bl_score
```

В данном случае, мы оценивает тестовую эффективность базовой модели по метрике R-квадрат и сохраняем ее в переменную для дальнейшего использования:

```
-1.6511340762242592
```

Базовая модель демонстрирует очень низкий уровень эффективности, хуже случайности, хуже предсказания среднего значения. Если бы мы продиагностировали ее, мы бы поняли, что проблема в очень высокой вариативности модели. Самая простая модель уже "переобучается" на наших данных. Это происходит потому, что в данных очень много признаков, каждый их которых добавляет одну степень свободы модели (добавляет один обучаемый коэффициент), что увеличивает её сложность. При этом далеко не все эти признаки нужны для предсказания значения целевой переменной. Как правило, при большом количестве признаков, большинство не несет полезной информации.

Еще будет полезно изобразить линию регрессии на графике, чтобы визуально убедиться в ее низком качестве:



Определение относительной важности признаков

Для улучшения работы модели нам нужно избавиться от неинформативных, лишних признаков в датасете. Существует множество стратегий, как можно это сделать. Например, мы можем исключать признаки по одному и следить, исключение каких лучше всего влияет на модель. Однако, эта стратегия, называемая "рекурсивное исключение признаков", будет слишком медленно работать, так как у нас больше 200 признаков и чтобы исключить хотя бы один, нам нужно обучить более 200 моделей и так далее.

Также можно попробовать рекурсивное добавление признаков. В этом случае мы выбираем признак, который дает наибольшую эффективность в задаче парной регрессии. После этого также перебором подбираем к нему второй и так далее. Этот способ будет ненамного быстрее и также потребует большого количества вычислительных ресурсов.

Можно использовать парные статистические критерии, которые оценивают степень взаимного влияния двух переменных. Это, например, хи-квадрат, тест Фишера для задач классификации, коэффициент корреляции для регрессии. В данном случае, можно построить коррелограмму, то есть матрицу коэффициентов парной корреляции. Или просто посчитать корреляцию каждого признака с целевой переменной. После этого останется только выбрать те признаки, у которых такой коэффициент выше.

Как мы говорили в лекциях, коэффициенты обученной модели линейной регрессии имеют очень схожий смысл. Можно посмотреть на коэффициенты обученной модели и выбрать те признаки, коэффициенты при который сильнее отличаются от 0. Однако, этот способ, как и оценка коэффициента корреляции, учитывает только линейную связь между конкретным признаком и целевой переменной.

Можно воспользоваться информацией, которую дает обученная нейлинейная модель. Мы уже говорили, что построение, например, дерева решений позволяет оценить относительную важность признаков. Причем эта важность будет учитывать не только линейное, но и более сложное нелинейное и совместное влияние факторов. Плюс, библиотека sklearn позволяет получить эту информацию автоматически, после обучения модели, нам не нужно специально что-то отдельно вычислять. Так можно узнать, какие из них оказывают наибольшее влияние на значение целевой переменной.

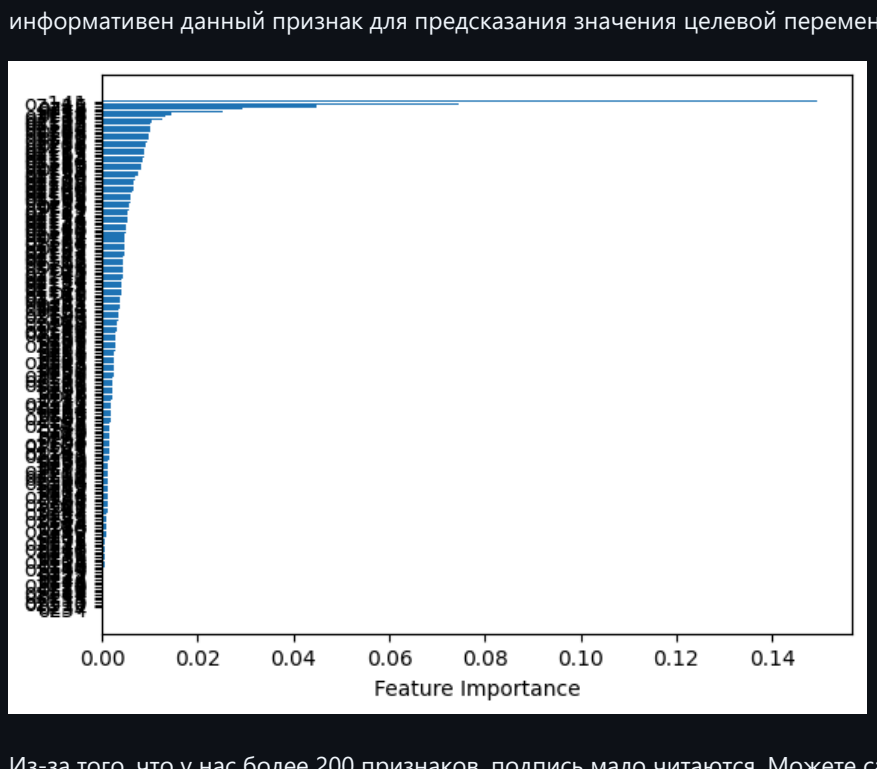
В данном примере мы используем даже не отдельное дерево, которое может очень сильно переобучиться на нашей выборке, а его более сильную и робастную ансамблевую версию - случайный лес:

```
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(n_estimators=15).fit(X_train, y_train)
```

По сути, случайный лес - это набор деревьев, каждый из которых видит случайную часть выборки. Более подробно по ансамбли моделей мы поговорим далее в курсе. Сейчас важно, что при помощи свойства feature_importances_ можно получить информацию о важности признаков. Удобнее всего изобразить эту информацию на графике в отсортированном виде:

```
sort = rf.feature_importances_.argsort()
plt.barh(df.data.columns[sort], rf.feature_importances_[sort])
plt.xlabel("Feature Importance")
```

Мы получим столбчатый график, на котором по вертикали отложены все наши признаки, они подписаны слева вдоль вертикальной оси, по горизонтали - отложена относительная важность данного признака. Это условное число, которое показывает, насколько информативен данный признак для предсказания значения целевой переменной:



Из-за того, что у нас более 200 признаков, подпись мало читаются. Можете самостоятельно построить более читаемый график, отобрав, например, только 30 самых значимых признаков. Мы можем просто вывести значимость самых важных колонок датасета:

```
rf.feature_importances_[sort][-10:]
```

Обратите внимание, из-за того, что для построения графика мы сортировали массив по возрастанию, самые значимые признаки - в конце. Таким образом, нам нужны, например, 10 последних элементов в этом массиве. Вот что получаем:

```
array([0.01009865, 0.01032001, 0.01242211, 0.01312193, 0.01436147,
       0.02522823, 0.02923168, 0.04488688, 0.07439497, 0.14930889])
```

Более интересна и полезна для нас информация о названиях самых важных признаков. Их тоже можно вывести:

```
df.data.columns[sort][-10:]
```

Мы получаем соответствующее количество признаков, оказывающих наибольшее влияние на целевую переменную. Обратите внимание, что самые важные признаки будут в конце списка. Но для дальнейших целей нам не важен порядок колонок.

```
Index(['oz160', 'oz155', 'oz197', 'oz137', 'oz158', 'oz18', 'oz35', 'oz48',
       'oz15', 'oz141'],
      dtype='object')
```

Теперь мы можем использовать эту информацию для удаления лишних данных из датасета. Количество самых важных признаков мы выбираем сами. Зачастую для этого используют "метод локтя". Можете самостоятельно попробовать разное количество признаков, мы сейчас возьмем 20:

```
trimmed = df.data[df.data.columns[sort][-20:]]
trimmed.head()
```

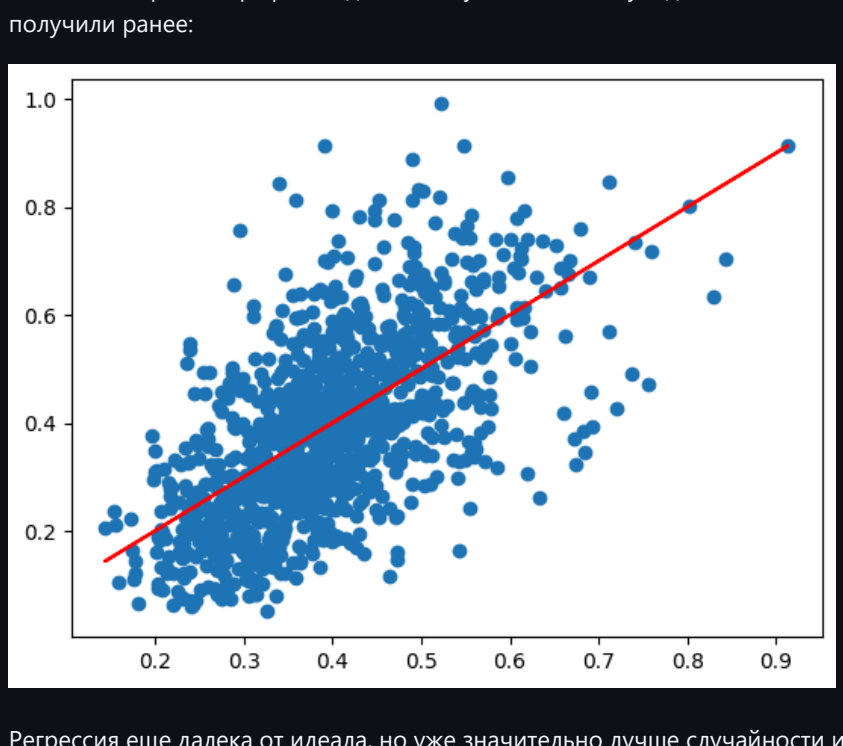
Не рекомендуется модифицировать исходную переменную, лучше создать новую копию датасета, в которую перенести только нужные колонки. Конечно, это нужно сделать и в обучающей и в тестовой части выборки. Либо, в исходном общем датасете, а затем повторить разбиение еще раз. Теперь все готово для того, чтобы построить модель на урезанном датасете:

```
X_train, X_test, y_train, y_test = train_test_split(trimmed, df.target, test_size=0.25, random_state=42)

better = LinearRegression()
better.fit(X_train, y_train)

print(bl_score)
better.score(X_test, y_test)
```

Мы используем тот же класс моделей - линейную регрессию - чтобы различия в метрике были сопоставимы. И мы получаем гораздо более качественную модель. Метрика уже положительна, что свидетельствует о большом росте точности:

```
-1.6511340762242592
0.3885997152790919
```

Можно изобразить график модели и визуально, чтобы убедиться в том, что он разительно отличается от графика, который мы получили ранее:



Регрессия еще далека от идеала, но уже значительно лучше случайности и показывает некоторый уровень эффективности, который может быть приемлемым в зависимости от прикладной задачи.

Мы, конечно, оценивали качество модели только по одной метрике. Можете самостоятельно сравнить значение других метрик качества регрессии на этих двух моделях. Убедитесь, что вторая модель лучше по любой выбранной метрике.

В данном примере сокращение количества столбцов в 10 раз, с 200 до 20, пошло только на пользу модели, так как избавило ее от лишних признаков, которые искусственно завышают сложность и вариативность функции гипотезы за счет введение большого числа коэффициентов. Другими словами, отбор признаков по важности оказывает регуляризирующее действие на модель.

Автоматизация отбора признаков

Конечно, отбор признаков - это довольно стандартная процедура при моделировании. В предыдущем примере мы все делали руками. Однако, в библиотеке sklearn есть встроенные средства выбора признаков. Познакомьтесь с ними в документации. Sklearn умеет автоматизировать как рекурсивное исключение и добавление признаков, и отбор по статистическим критериям, так и отбор по результатам обучения модели.

Давайте напомним себе о форме датасета:

```
(4450, 202)
```

Исходно у нас присутствует 202 признака. В sklearn есть специальный объект, SelectFromModel, который находится в пакете feature_selection. Познакомьтесь с документацией к этому классу и к другим классам из данного пакета. Работа с ними напоминает работу с другими классами преобразования данных в том плане, что используется подход fit-transform:

```
from sklearn.feature_selection import SelectFromModel

sfm = SelectFromModel(RandomForestRegressor(n_estimators=15)).fit(df.data, df.target)
X_trimmed = sfm.transform(df.data)
X_trimmed.shape
```

Обратите внимание, что мы передаем в этот объект вид модели машинного обучения, на основе которой будет производиться отбор признаков. При этом, методы fit() объекта SelectFromModel в том числе произведет обучение модели. Помните об этом, если используете ресурсоемкую модель. Данный объект можно настроить для использование уже обученной модели, без повторного запуска обучения.

Также обратите внимание, что мы используем данное преобразование на всем датасете. В дальнейшем мы опять разобьем его на обучающую и тестовую выборки. Однако, так как объект преобразования SelectFromModel сохраняется и "запоминает" нужные признаки, мы можем обучить его и уже после разбиения. В таком случае, метод transform() нужно будет вызвать и для обучающей и для тестовой выборки отдельно. Преобразование будет скоординированным.

Посмотрим, как данный код преобразовал данные:

```
(4450, 55)
```

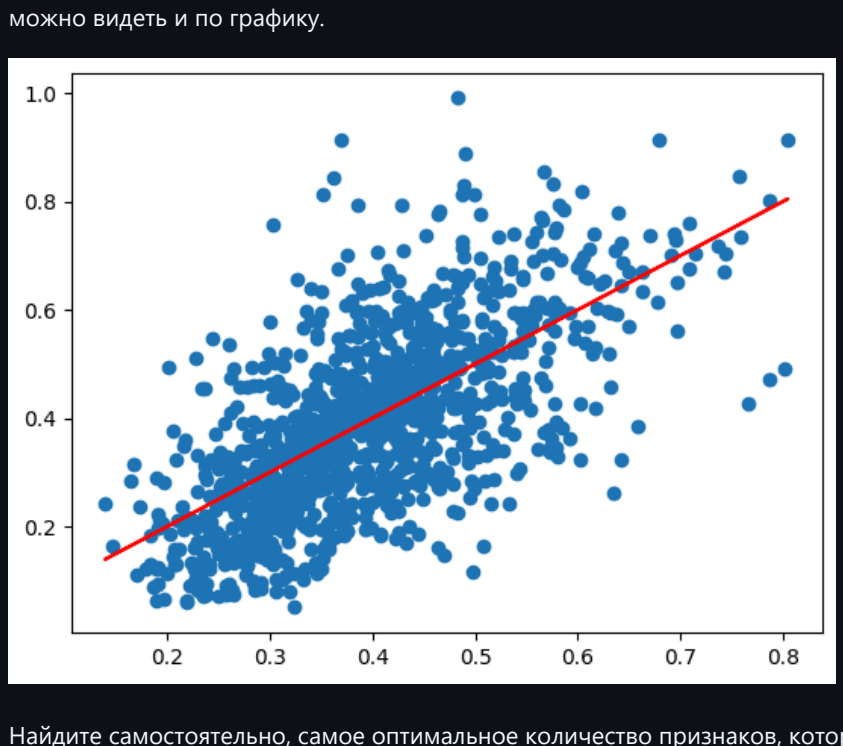
У нас осталось 55 признаков. Это больше, чем мы использовали в прошлый раз. Решение об этом принимает сам алгоритм SelectFromModel. Его, конечно, тоже можно настроить. Но сейчас давайте проверим, как данная обработка скажется на эффективности модели. Для этого построим уже третью модель на этом датасете:

```
X_train, X_test, y_train, y_test = train_test_split(X_trimmed, df.target, test_size=0.25, random_state=42)

better = LinearRegression()
better.fit(X_train, y_train)

print(bl_score)
better.score(X_test, y_test)
```

Сы видим еще большее улучшение по метрикам:

```
-1.6511340762242592
0.4314653462618252
```

Опять же, самостоятельно оцените данную модель и по другим метрикам регрессии. Особенно интересно ее сравнение со второй моделью. Но по метрика R-квадрат мы видим, что увеличение количества признаков до 55 не привело к переобучению модели. То же можно видеть и по графику.



Найдите самостоятельно, самое оптимальное количество признаков, которые следует оставить в модели для достижения наиболее высокой тестовой эффективности.

Устранение дисбаланса классов

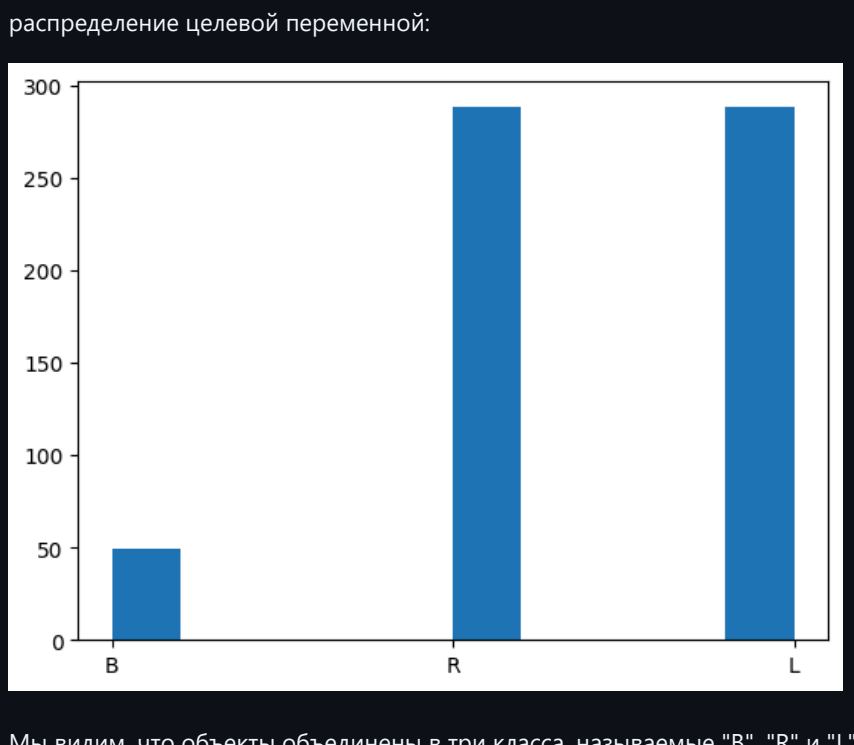
Одна из самых частых проблем при построении моделей классификации на реальных данных - дисбаланс классов. Это ситуация, когда в датасете присутствует очень разное количество объектов, принадлежащих разным классам. Другими словами, это неравномерность распределения значений целевой переменной. При этом проблемой такой дисбаланс становится, когда объектов одного класса в несколько раз, а то и десятков раз больше чем другого. На практике, соотношение, например, 100 к 1 не является редкостью.

Это может быть проблемой при обучении моделей потому, что при подборе весов модель будет чаще видеть в обучающих примерах объекты мажоритарного или мажоритарных классов (то есть таких, объектов которых значительно больше). И их влияние на изменение параметров функции гипотезы будет превалировать над влиянием объектов миноритарных классов. В итоге модель может одновременно недообучиться на мажоритарных классах и переобучиться на миноритарных. При анализе результатов моделирования эту проблему проще всего выявить при рассмотрении отчета о классификации - объекты миноритарных классов распознаются значительно хуже.

Давайте обратимся к примеру. Возьмем датасет результатов психологического моделирования:

```
df = fetch_openml("balance-scale", version=1)

df.data.head()
```

Этот набор данных не требует технической предварительной обработки для построения модели, поэтому сразу визуализируем распределение целевой переменной:

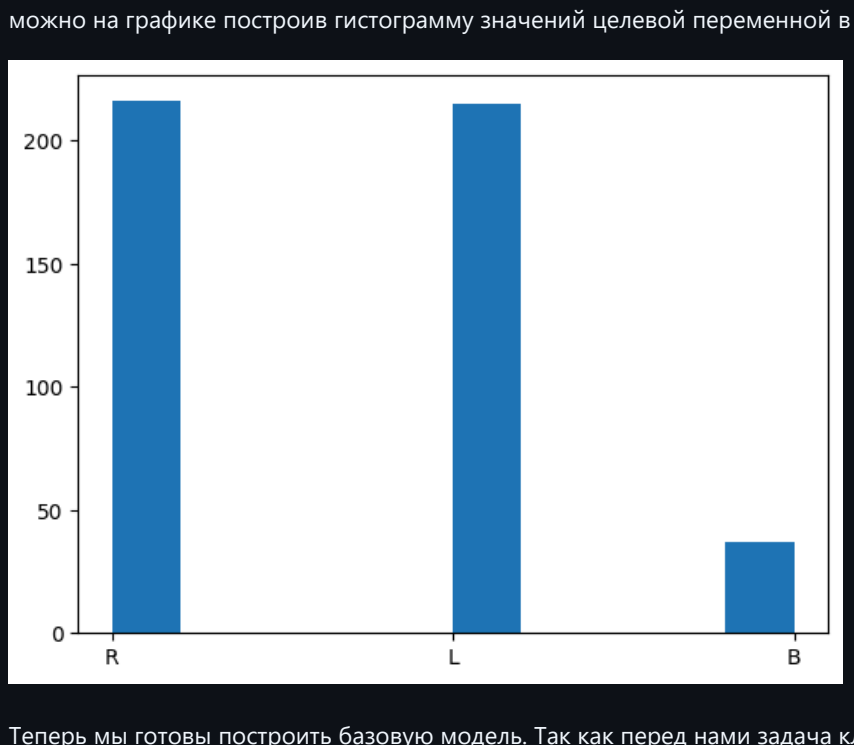


Мы видим, что объекты объединены в три класса, называемые "B", "R" и "L". Причем, объектов класса "B" примерно в шесть раз меньше, чем каждого из двух других. Это - миноритарный класс, а два других - мажоритарные. Для построения базовой модели, как всегда разделим выборку:

```
X_train, X_test, y_train, y_test = train_test_split(df.data, df.target, test_size=0.25, random_state=42, stratify=df.target)
```

Обратите внимание, что при разделении несбалансированных выборок за счет случайных ошибок мы можем существенно сместить форму этого распределения. Вполне вероятно, что объекты миноритарного класса вообще по прихоти случайности не попадут в тестовую выборку, или попадут в еще меньшей доле. Ил наоборот. Этого следует избегать, так как для адекватного оценивания качества модели, обучающая и тестовая выборки должны быть как можно более сходными по своим статистическим характеристикам. Все статистики в выборке мы проконтролировать не можем, но хотя бы должны убедиться, что распределение целевой переменной будет сходным.

Для этого применяют особый прием - стратификацию выборки. В функцию train_test_split можно передать специальный аргумент, который заставит ее учитывать распределение объектов по данной переменной. В нашем случае, мы используем целевую переменную для стратификации. Это гарантирует, что две подвыборки после разделения сохранять форму данного распределения. Убедиться можно на графике построив гистограмму значений целевой переменной в одной из выборок после разделения:



Теперь мы готовы построить базовую модель. Так как перед нами задача классификации, в качестве базовой выберем логистическую регрессию:

```
baseline = LogisticRegression()
baseline.fit(X_train, y_train)
bl_score = baseline.score(X_test, y_test)
bl_score
```

Базовая модель дает примерно 86% точности:

```
0.8598726114649682
```

Опять же, можно оценивать модель и по другим метрикам. Более того, постановка задачи будет диктовать нам, какая метрика будет целевой, то есть на какую метрику мы должны ориентироваться в первую очередь при выборе и оценке моделей.

Так как мы исследуем проблему дисбаланса классов, обязательно надо построить отчет о классификации:

```
from sklearn.metrics import classification_report, confusion_matrix

print(classification_report(y_test, baseline.predict(X_test)))
```

На нем мы видим, что эффективность модели действительно разнится для разных классов:

```
              precision    recall  f1-score   support

           B       0.00      0.00      0.00        12
           L       0.87      0.92      0.89        73
           R       0.86      0.94      0.90        72

    accuracy                           0.86       157
   macro avg       0.58      0.62      0.60       157
weighted avg       0.80      0.86      0.83       157
```

Еще больше информации дает вывод матрицы классификации:

```
print(confusion_matrix(y_test, baseline.predict(X_test)))
```

Фактически, модель вообще ни разу не распознала правильно объект миноритарного класса. Поэтому невзвешенная оценка качества модели на самом деле ближе к отметке в 60%. Это не очень удовлетворительный результат.

```
[[ 0  6  6]
 [ 1 67  5]
 [ 0  4 68]]
```

Давайте посмотрим, удастся ли нам улучшить данную модель. Как всегда, есть несколько стратегий борьбы с дисбалансом классов. Здесь рассмотрим два: взвешивание классов и ресемплинг выборки.

Главная проблема дисбаланса классов в том, что модель недостаточно учитывает объекты миноритарных классов. Можно относительно просто это исправить, при обучении модели, придав больший "вес" таким редким объектам. Это называется, взвешивание классов. Для начала надо рассчитать "важность" или вес класса, который будет обратно пропорционален его доле в выборке. Можно сделать это и руками, но в библиотеке sklearn есть встроенная функция для этого:

```
from sklearn.utils.class_weight import compute_class_weight

class_weights = compute_class_weight('balanced', classes=np.unique(y_train), y=y_train)
class_weights = dict(zip(np.unique(y_train), class_weights))
class_weights
```

Предпоследняя строчка в этом коде нужна, чтобы правильно отформатировать получившиеся значения в словарь, где ключами являются метки классов, а значениями - вес соответствующего класса:

```
{'B': 4.216216216216216, 'L': 0.7255813953488373, 'R': 0.7222222222222222}
```

Мы видим, что вес миноритарного класса действительно вильно больше, чем двух других. Эту информацию можно передать непосредственно модели машинного обучения через параметр конструктора:

```
weighted = LogisticRegression(class_weight=class_weights)
weighted.fit(X_train, y_train)
print(bl_score)
weighted.score(X_test, y_test)
```

Такая модель будет сильнее изменять свои веса в ответ на объект миноритарного класса, пропорционально весу этого класса. На самом деле все немного сложнее, так как обучение идет методом пакетного градиентного спуска, в котором на каждом шаге обрабатывается несколько объектов выборки, но смысл именно в этом.

Оценим качество такой модели на отчете о классификации:

```
              precision    recall  f1-score   support

           B       0.61      0.92      0.73        12
           L       0.97      0.92      0.94        73
           R       0.96      0.93      0.94        72

    accuracy                           0.92       157
   macro avg       0.85      0.92      0.87       157
weighted avg       0.94      0.92      0.93       157
```

Мы видим, что точность модели возросла с 86% до 92%. Это очень существенное увеличение эффективности. Причем обратите внимание, что оценивали модель мы на той же тестовой выборке, в которой сохранено исходной неравномерное распределение. То есть это естественное повышение качества. При этом точность модели на этом миноритарном классе все равно немного ниже, чем на других. Это тоже естественно. Но разница уже далеко не такая огромная.

Oversampling

Но что делать, если конкретная модель не поддерживает взвешивание классов? Или по какой-то другой причине, такой способ либо не подходит, либо не дает нужного эффекта? Можно использовать ресемплинг - то есть случайную выборку из исходного датасета с выравниванием распределения по классам. Есть две стратегии. Оверсемплинг - это когда мы семплируем в выборку больше объектов мажоритарного класса с повторениями. Андерсемплинг - это исключение случайных объектов мажоритарных классов до выравнивания распределения. Есть гибридный подход - когда мы делаем и то и другое в определенных пропорциях. Кроме того, есть продвинутые техники генерации или аугментации данных для выравнивания распределения.

Покажем на простом примере как работает оверсемплинг. Подсчитаем точное количество объектов каждого класса в обучающей выборке:

```
y_train.value_counts()
```

Видим, что объектов миноритарного класса всего 37 против 216 - мажоритарного:

```
R    216
L    215
B     37
```

Для дальнейших манипуляций нам будет удобно объединить матрицу признаков и вектор целевой переменной в один датафрейм:

```
X_train["target"] = y_train
```

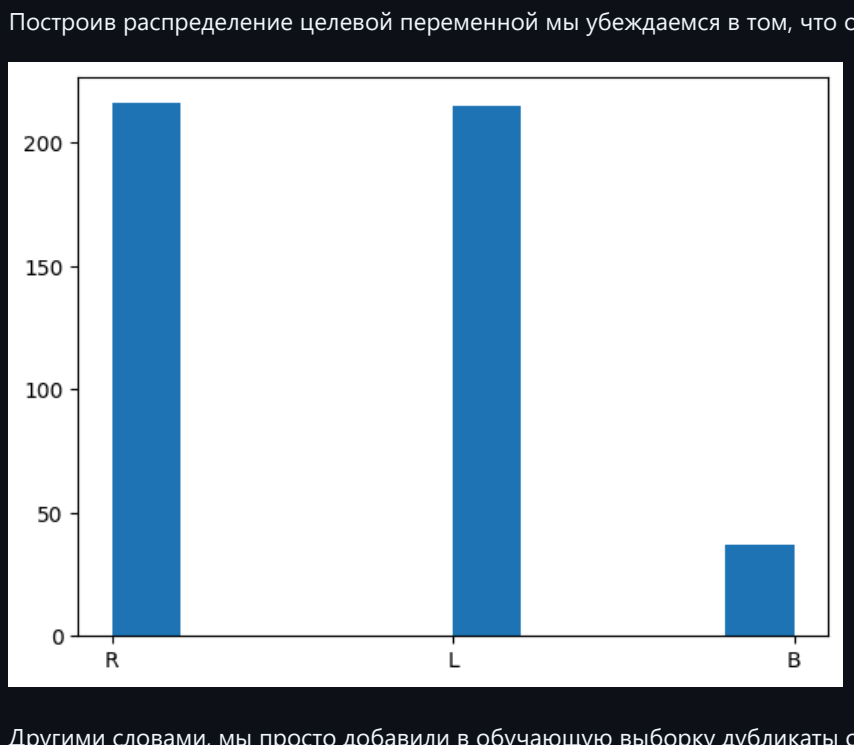
Теперь мы семплируем недостающее количество объектов миноритарного класса из нашего датасета с повторениями:

```
oversampled = X_train[X_train.target == "B"].sample(n=216-37, replace=True, ignore_index=True)
```

После этого нам остается только объединить эту новую выборку с исходной:

```
oversampled = pd.concat([X_train, oversampled])
print(oversampled.shape)
oversampled.head()
```

Построив распределение целевой переменной мы убеждаемся в том, что оно стало очень близко к равномерному:



Другими словами, мы просто добавили в обучающую выборку дубликаты объектов миноритарного класса в нужном объеме. В итоге, по задумке этого метода, при обучении модель будет чаще видеть такие объекты (пусть одни и те же) и будет подстраивать свои веса под них в том же темпе, что и под объекты мажоритарных классов. Проверим это. Опять разделим датафрейм на матрицу признаков и целевой вектор:

```
y_train_OS = oversampled.target
X_train_OS = oversampled.drop(["target"], axis=1)
```

И построим такую же модель логистической регрессии. Теперь никакое взвешивание классов не понадобится, мы сделали все руками:

```
OSmodel = LogisticRegression()
OSmodel.fit(X_train_OS, y_train_OS)
print(bl_score)
OSmodel.score(X_test, y_test)
```

При оценке качества этой модели получаем такую же картину, что и после взвешивания:

```
              precision    recall  f1-score   support

           B       0.61      0.92      0.73        12
           L       0.97      0.92      0.94        73
           R       0.96      0.93      0.94        72

    accuracy                           0.92       157
   macro avg       0.85      0.92      0.87       157
weighted avg       0.94      0.92      0.93       157
```

Данные стратегии производят очень схожий эффект на процесс обучения модели.

Дискретизация целевой переменной

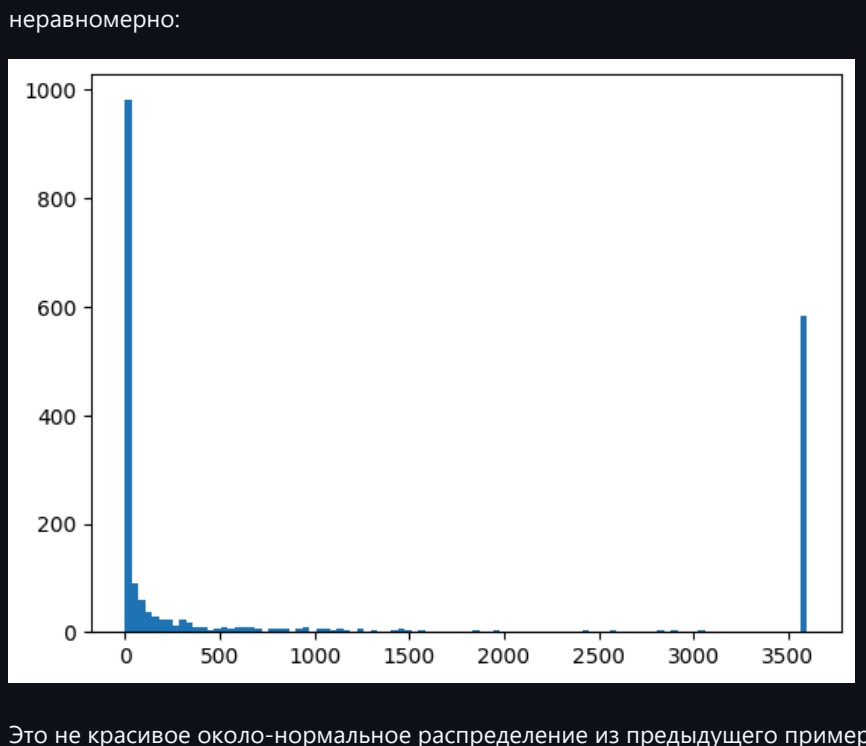
Для освоения следующего приема обработки данных воспользуемся другим датасетом, но из того же репозитория. После чтения сразу подготовим его к моделированию:

```
df = fetch_openml("CPMP-2015-regression", version=1)
df.data.drop(["instance_id"], inplace=True, axis=1)
df.data = pd.get_dummies(df.data)
df.data.head()
```

Это набор данных о бенчмарке решения математической проблемы нахождения оптимальной сортировки на контейнерной площадке. В этом наборе нам более всего важно распределение целевой переменной:

```
plt.hist(df.target, 100)
_ = plt.plot()
```

Мы видим, что целевая переменная имеет численный вид (то есть перед нами проблема регрессии), и при этом, распределена очень неравномерно:



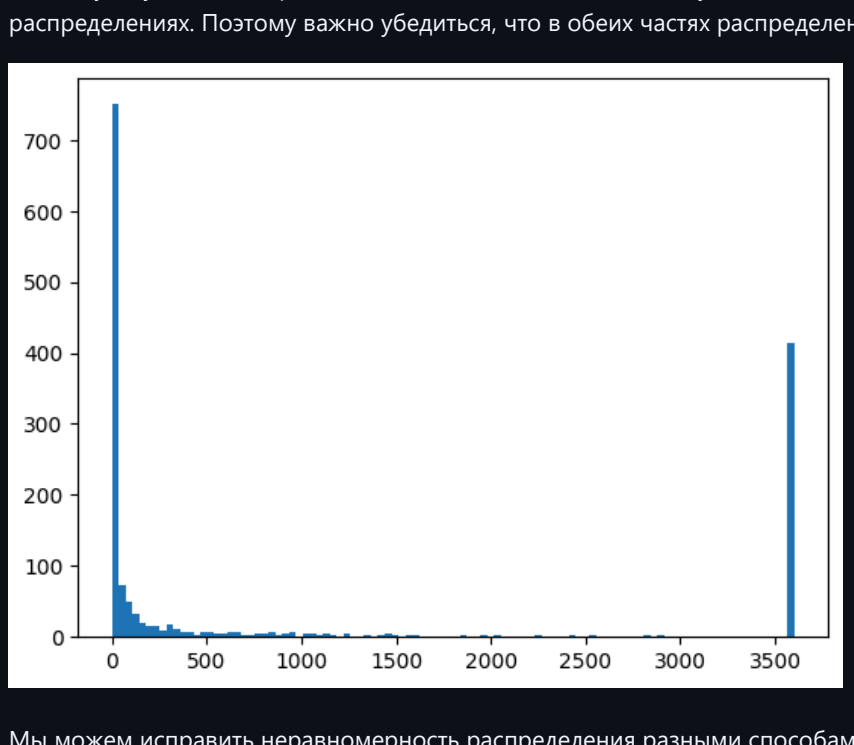
Это не красивое около-нормальное распределение из предыдущего примера. Это очень очень несбалансированное двухмодальное распределение. Есть большое количество объектов в датасете, у которых значение целевой переменной близко к 0, чуть меньшее, но тоже большое количество объектов, у которых оно близко к максимальному (порядка 3 500), и очень мало объектов с промежуточными значениями.

Такое странное неравномерное распределение может повлечь проблемы при моделировании. Проблема сходная с дисбалансом классов, но для регрессии. Модель будет при своем обучении очень часто видеть объекты со схожим значениями целевой переменной и очень редко - объекты с промежуточными. В итоге модель может не научится распознавать такие промежуточные объекты.

Покажем, как можно уменьшить влияние такого распределения. Но сначала разделим датасет на обучающую и тестовую выборки, чтобы отдельно обработать их уже после разделения:

```
X_train, X_test, y_train, y_test = train_test_split(df.data, df.target, test_size=0.25, random_state=42)
```

Как мы уже упоминали, разделение датасета может внести случайный ошибки выборки, что особенно критично при неравномерных распределениях. Поэтому важно убедиться, что в обеих частях распределение сохранило свою форму, хотя бы в общих чертах:



Мы можем исправить неравномерность распределения разными способами, среди которых также ресемплирование выборки, как и в случае с классификацией. Но здесь покажем другой подход - дискретизацию целевой переменной. Этот способ заключается в том, что мы объединяем значения целевой переменной в категории - bins - поэтому такой способ часто называют биннинг.

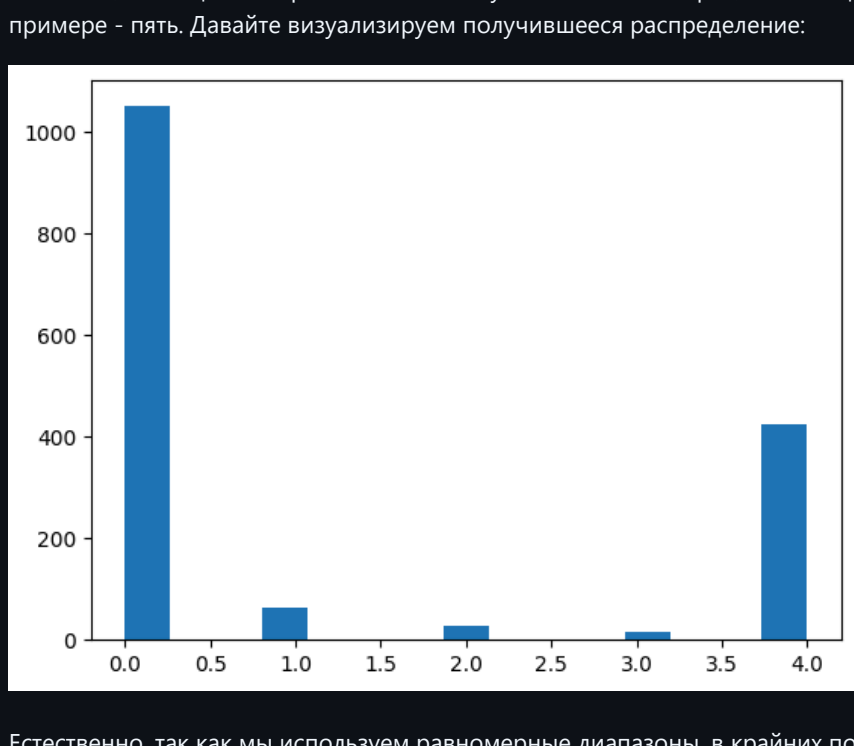
Группировать объекты можно опять же по-разному. Для автоматизации этой процедуры в библиотеке sklearn есть специальный объект - KBinsDiscretizer, который находится в пакете preprocessing. Воспользуемся им и создадим, например, пять групп:

```
from sklearn.preprocessing import KBinsDiscretizer

y_binned = KBinsDiscretizer(n_bins=5, encode="ordinal", strategy="uniform").fit_transform(pd.DataFrame(y_train))
```

Группировка объектов означает, что мы переходим к категориальному типу в целевой переменной. Каждая категория будет обозначать некоторый диапазон исходных значений. При этом категории нумеруются последовательно, как при применении OrdinalEncoder при кодировании категориальных признаков. Но в этом случае, применение такой кодировки оправдано, так как эти категории, обозначая диапазоны значений численной шкалы, имеют естественный порядок. Вот как выглядят значения целевой переменной после дискретизации:

```
array([[3.],
       [2.],
       [3.],
       ...,
       [2.],
       [1.],
       [3.]])
```

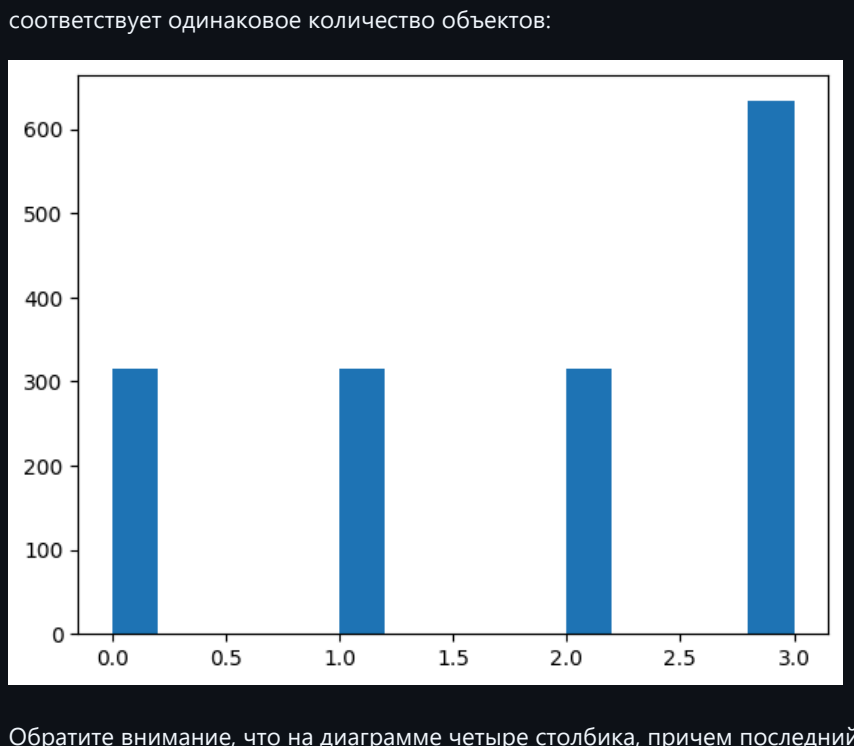
Самая простая стратегия биннинга, которая применяется в этом объекте по умолчанию - равномерная. Она делит существующий диапазон значений на указанное количество равных по величине поддиапазонов, каждый из которых кодируется последовательным натуральным числом. По сути, мы просто укрупняем целевую переменную. Если раньше у нас было произвольно большое количество значений, потенциально равное количеству объектов в выборке, то после дискретизации остается указанное количество, в нашем примере - пять. Давайте визуализируем получившееся распределение:



Естественно, так как мы используем равномерные диапазоны, в крайних поддиапазонах соберутся большинство объектов выборки. Мы это видим на графике - столбцы укрупнились, но неравномерность распределения никуда не делась. В случае таких сильно неравномерных распределений нам больше подойдет другая стратегия - квантильная дискретизация, при которой длина диапазонов выбирается таким образом, чтобы в каждом из них оказалось примерно по одинаковому количеству объектов выборки:

```
y_binned = KBinsDiscretizer(n_bins=5, encode="ordinal", strategy="quantile").fit_transform(pd.DataFrame(y_train))
```

Такой способ полностью исключит неравномерность распределения. На графике хорошо видно, что теперь каждой категории соответствует одинаковое количество объектов:



Обратите внимание, что на диаграмме четыре столбика, причем последний в два раза выше. Это всего лишь артефакт визуализации, в которой две последние категории объединились. Самостоятельно поэкспериментируйте с визуализацией и выберите такой график, который корректно показывает получившееся распределение.

Задания для самостоятельного выполнения

1. Исследуйте связь между количеством самых важных признаков, которые использует модель для обучения и тестовой точностью получившейся модели. Обучите несколько моделей с разным количеством наиболее важных признаков. Постройте график зависимости точности модели от количества признаков. Сделайте вывод.
2. Используйте другие методы отбора признаков:
   i. Исключение низкодисперсных признаков;
   ii. Исключение по парным стаистическим критериям (хи-квадрат, тест Фишера, коэффициент корреляции, информационный критерий);
   iii. Рекурсивное исключение признаков;
   iv. Последовательное включение признаков;
   v. Исключение по L1-норме (гребневой регрессии).
3. Изучите возможности библиотеки imbalanced-learn. Примените на данном примере возможности данной библиотеки для оверсемплинга и андерсемплинга выборки.
4. Исследуйте влияние дискретизации целевой переменной на качество модели. Используйте уже продемострированный подход - построение базовой модели (baseline) и сравнение модели после обработки данных с базовой. Проверьте разное количество категорий, а также разные стратегии группировки. Сделайте выводы. Обратите внимание, что после биннинга целевой переменной она стала категориальной. А значит, задача превратилась в задачу классификации.

Контрольные вопросы

1. Какие модели лучше всего можно использовать для отбора признаков? Почему другие нельзя или нежелательно?
2. Зачем нужен этап отбора признаков? В каких случаях без него не обобйтись? А в каких его можно пропустить?
3. Какие есть методы отбора признаков? Найдите и опишите не менее пяти.
4. Какие есть стратегии устранения дисбаланса классов? В каких случаях стоит применять их и от чего зависит выбор стратегии?
5. Какие модели машинного обучения из библиотеки sklearn поддерживают веса классов?
6. Зачем использовать дискретизацию непрерывной целевой переменной? В каких случаях это оправданно, а в каких - нет?
7. Почему дискретизацию целевой переменной нужно делать только после разделения на тестовую и обучающую подвыборки? Что такое утечка данных?

Дополнительные задания

1. Повторите приведенный в данной работе анализ полностью на другом датасете. Сделайте вывод.
2. Используйте продвинутые алгоритмы дискретизации целевой переменной, например, CART.
3. Оформите алгоритм обработки данных как конвейер (pipeline) sklearn.
4. Изучите и примените продвинутые стратегии оверсемплинга выборки: SMOTE, ASMO, ADASYN. Поясните механизм их работы и применимость в разных задачах.
5. Изучите и примените метод андерсемплинга выборки, основанный на Tomek Links. Сделайте вывод о его применимости к разным задачам.
6. Для датасета по классификации модифицируйте предсказание второй модели так, чтобы вернуть постановку задачи регрессии. Для этого каждой категории присвойте численное значение. Это можно сделать, вычислив, например, медиану. Теперь можно считать, что модель предсказывает не метку категории, а конкретное численное значение. Как следствие, для оценки такой модели можно использовать метрики качества регрессии. Сравните метрики до и после преобразования.